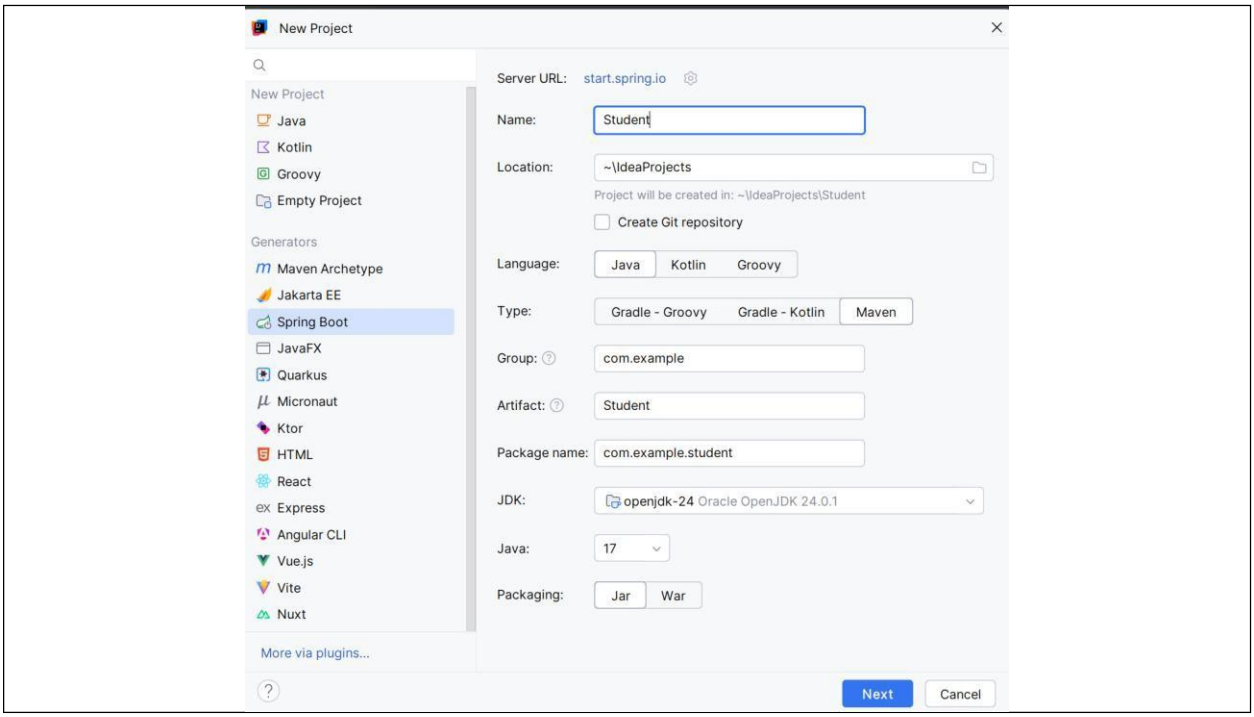
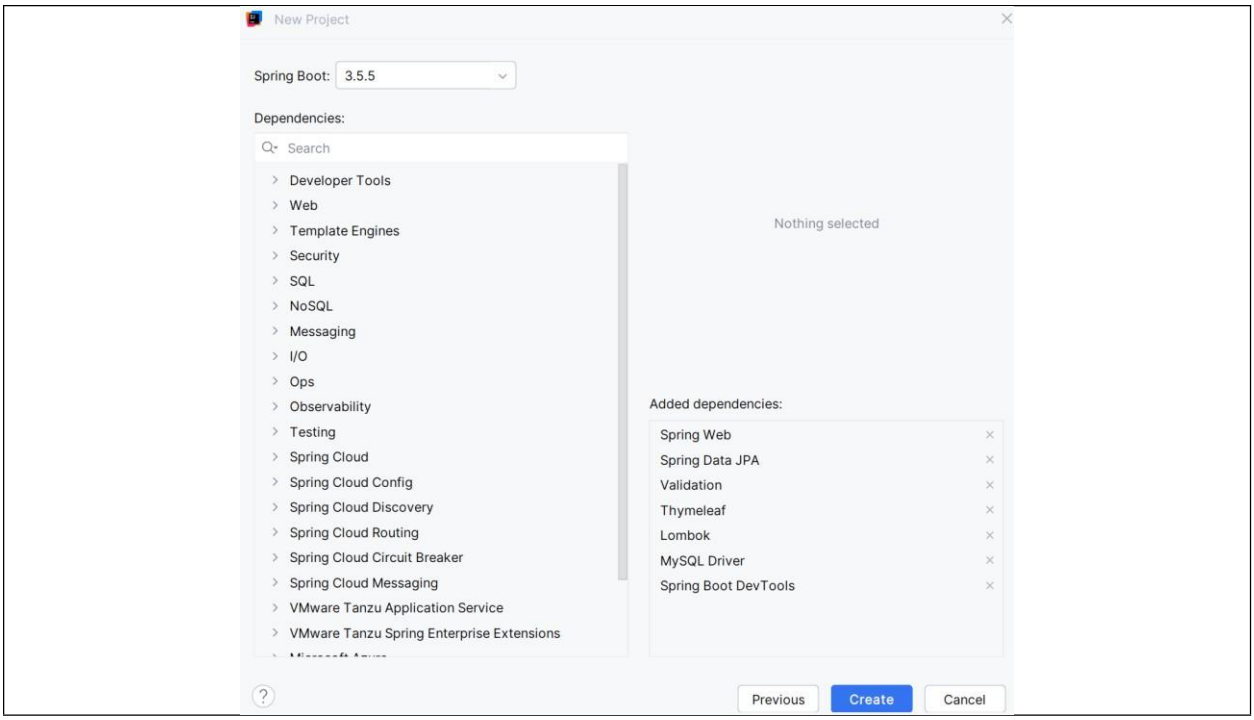
Lab 5- Java 17+, Spring Boot 3.x, IntelliJ, XAMPP/MySQL (running), Maven (Student project).

1. Open XAMPP and run MySQL server.
2. Open IntelliJ and create a new project Named the project student. Make sure you choose Maven and Jar.



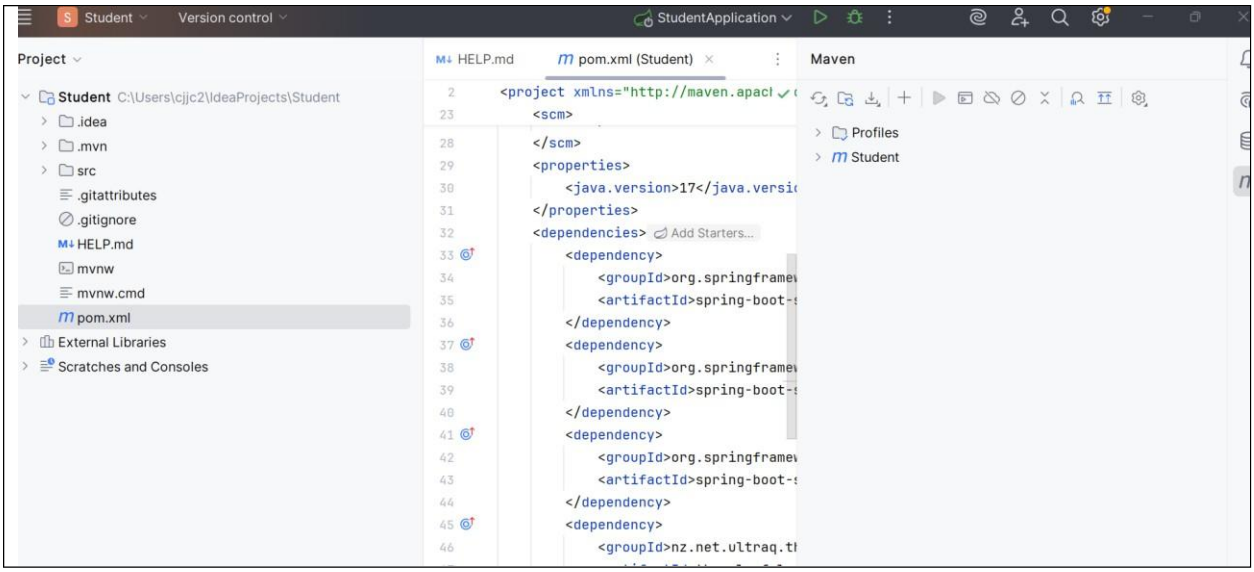
3. Click next and add the following dependences.



4. Click create. Check your POM file. The POM file dependencies are automatically generated at the beginning however there are some dependencies that might not be available. Compare and add those dependencies. Click sync to download that dependency.

```
<dependencies>
  <!-- Web / MVC -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>

  <!-- Thymeleaf -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-thymeleaf</artifactId>
  </dependency>
  <dependency>
    <groupId>nz.net.ultraq.thymeleaf</groupId>
    <artifactId>thymeleaf-layout-dialect</artifactId>
  </dependency>

  <!-- JPA + MySQL -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
```

```
  </dependency>
  <dependency>
    <groupId>com.mysql</groupId>
    <artifactId>mysql-connector-j</artifactId>
    <scope>runtime</scope>
  </dependency>

  <!-- Validation -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-validation</artifactId>
  </dependency>

  <!-- Lombok (dev only) -->
  <dependency>
    <groupId>org.projectlombok</groupId>
    <artifactId>lombok</artifactId>
    <optional>true</optional>
  </dependency>

  <!-- WebJars: Bootstrap (choose WebJars OR CDN; here we use WebJars only) -->
  <dependency>
    <groupId>org.webjars</groupId>
    <artifactId>bootstrap</artifactId>
    <version>5.3.3</version>
  </dependency>

  <!-- DevTools (optional) -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-devtools</artifactId>
    <scope>runtime</scope>
  </dependency>
</dependencies>
```



5. Go to the application properties and prepare this portion to connect to MySQL Note that we change the server port to 8082. This means that when you try to render this program, you should use 8082.

```
server.port=8082

spring.datasource.url=jdbc:mysql://localhost:3306/st_db?createDatabaseIfNotExist=true&useSSL=false&allowPublicKeyRetrieval=true&serverTimezone=UTC
spring.datasource.username=root
spring.datasource.password=

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true

spring.thymeleaf.cache=false
```

6. Create the following packages under the com.example.student (or your src folder)

7. Create the entity class under the model folder. Name it as Student.

```java
package com.example.student.model;

import jakarta.persistence.*;
import jakarta.validation.constraints.*;
import lombok.*;
import org.springframework.format.annotation.DateTimeFormat;

import java.util.Date;

@Data @NoArgsConstructor @AllArgsConstructor
@Entity
public class Student {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @NotBlank(message = "Name is required")
    @Size(max = 100, message = "Name must be at most 100 chars")
    private String name;

    @Temporal(TemporalType.DATE)
    @DateTimeFormat(pattern = "yyyy-MM-dd")
    @Past(message = "Date of birth must be in the past")
    private Date dob;

    private boolean passed;

    @DecimalMin(value = "0.0", message = "GPA must be >= 0.0")
    @DecimalMax(value = "4.33", message = "GPA must be <= 4.33")
    private Double gpa;
}
```

8. Create a repository class in side repositories. Name it as StudentRepository. This will extend the JpaRepository interface.

```
package com.example.student.respository;


import com.example.student.model.Student;

import org.springframework.data.domain.Page;
import org.springframework.data.jpa.repository.JpaRepository;

import org.springframework.data.domain.Pageable;

import java.util.List;


public interface StudentRepository extends JpaRepository<Student, Long> {

   List<Student> findByNameContainingIgnoreCase(String name);

}
```

9. Create the services class. This class can be incorporated with the controller if you want to. But to create a clean service layer separation, we will separate it into a different Java file. Make sure you add your own package at the top of this code.

```
package com.example.student.web;


import com.example.student.model.Student;


import com.example.student.respository.StudentRepository;
import lombok.RequiredArgsConstructor;
import org.springframework.data.domain.Pageable;  // keep this one

import org.springframework.data.domain.Page;
import org.springframework.data.domain.PageImpl;
import org.springframework.data.domain.PageRequest;
import org.springframework.data.domain.Pageable;
import org.springframework.data.domain.Sort;
import org.springframework.stereotype.Service;




import java.util.List;
import java.util.Optional;
@Service
@RequiredArgsConstructor
public class StudentService {
    private final StudentRepository repo;

    public List<Student> search(String keyword) {

        if (keyword == null || keyword.isBlank()) {
            return repo.findAll();
        }
```

```
        return repo.findByNameContainingIgnoreCase(keyword);
    }


    public Student get(Long id) { return repo.findById(id).orElseThrow();
}

    public Student save(Student s) { return repo.save(s); }

    public void delete(Long id) { repo.deleteById(id); }
}
```

10. Inside the web, create a StudentController class.

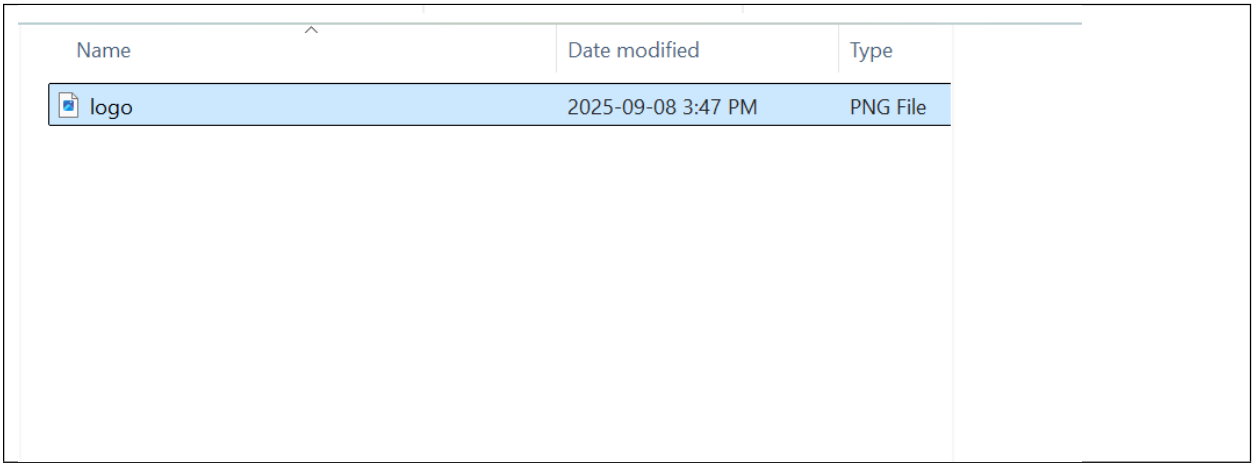
```
package com.example.student.web;


import com.example.student.model.Student;
import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.validation.BindingResult;
import org.springframework.web.bind.annotation.*;
import org.springframework.web.servlet.mvc.support.RedirectAttributes;

@Controller
@RequestMapping
@RequiredArgsConstructor
public class StudentController {
    private final StudentService service;

    // Redirect root to list
    @GetMapping("/")
    public String home() { return "redirect:/students"; } // replaces separate
/index default

    // List + search + pagination (keeps your /index but modernizes under
/students)
    @GetMapping({"/students", "/index"})
    public String list(
            @RequestParam(name = "keyword", required = false) String keyword,
            @RequestParam(defaultValue = "0") int start,
            @RequestParam(defaultValue = "10") int limit,
            Model model
    ) {

        var students = service.search(keyword);

        int end = Math.min(start + limit, students.size());
        var pageStudents = students.subList(start, end);

        model.addAttribute("students", pageStudents);
        model.addAttribute("keyword", keyword == null ? "" : keyword);
        model.addAttribute("start", start);
```

```
        model.addAttribute("limit", limit);
        model.addAttribute("total", students.size());

        return "students";
    }

    // Show create form
    @GetMapping("/students/new")
    public String createForm(Model model) {
        model.addAttribute("student", new Student());
        return "student-form";
    }

    // Persist (create)
    @PostMapping("/students")
    public String create(@Valid @ModelAttribute("student") Student student,
                         BindingResult br,
                         RedirectAttributes ra) {
        if (br.hasErrors()) return "student-form";
        service.save(student);
        ra.addFlashAttribute("success", "Student created successfully.");
        return "redirect:/students";
    }

    // Show edit form
    @GetMapping("/students/{id}/edit")
    public String editForm(@PathVariable Long id, Model model) {
        model.addAttribute("student", service.get(id));
        return "student-form";
    }

    // Persist (update)
    @PostMapping("/students/{id}")
    public String update(@PathVariable Long id,
                         @Valid @ModelAttribute("student") Student student,
                         BindingResult br,
                         RedirectAttributes ra) {
        if (br.hasErrors()) return "student-form";
        student.setId(id);
        service.save(student);
        ra.addFlashAttribute("info", "Student updated successfully.");
        return "redirect:/students";
    }

    // Delete via POST (safer than GET)
    @PostMapping("/students/{id}/delete")
    public String delete(@PathVariable Long id, RedirectAttributes ra) {
        service.delete(id);
        ra.addFlashAttribute("warning", "Student deleted.");
        return "redirect:/students";
    }
}
```

11. You are now ready to create the HTML templates. Create an HTML file called layout inside the template folder.

```
<!doctype html>
<html lang="en" xmlns:th="http://www.thymeleaf.org"
      xmlns:layout="http://www.ultraq.net.nz/thymeleaf/layout">
<head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Student App</title>

    <!-- if you are using Webjars make sure this is installed then remove
the cdn -->
    <link rel="stylesheet"
href="/webjars/bootstrap/5.3.3/css/bootstrap.min.css">

    <!-- HEAD (CSS), if you will use CDN you can now remove the webjars -->
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.cs
s" rel="stylesheet">

    <!-- END OF BODY (JS) -->
    <script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.m
in.js"></script>

</head>
<body>
<nav class="navbar navbar-expand-sm bg-dark navbar-dark">
    <div class="container-fluid">
        <a class="navbar-brand" th:href="@{/students}">
            <img th:src="@{/logo.png}" alt="logo" width="40" height="40">
        </a>
        <button class="navbar-toggler" type="button" data-bs-
toggle="collapse" data-bs-target="#nav">
            <span class="navbar-toggler-icon"></span>
        </button>
        <div id="nav" class="collapse navbar-collapse">
            <ul class="navbar-nav">
                <li class="nav-item"><a class="nav-link"
th:href="@{/students}">Home</a></li>
                <li class="nav-item"><a class="nav-link"
th:href="@{/students/new}">Add Student</a></li>
            </ul>
        </div>
    </div>
</nav>

<main class="container py-3" layout:fragment="content"></main>

<script src="/webjars/bootstrap/5.3.3/js/bootstrap.bundle.min.js"></script>
</body>
</html>
```

12. Put a logo inside your static folder. Choose any logo.



13. Now create the students.html.

```
<!doctype html>
<html lang="en" xmlns:th="http://www.thymeleaf.org"
      xmlns:layout="http://www.ultraq.net.nz/thymeleaf/layout"
      layout:decorate="layout">

<!-- HEAD (CSS) -->
<link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css
" rel="stylesheet">

<!-- END OF BODY (JS) -->
<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.mi
n.js"></script>

<head><meta charset="UTF-8"><title>Students</title></head>
<body>
<section layout:fragment="content">
    <!-- Flash alerts -->
    <div th:if="${success}" class="alert alert-success alert-dismissible fade
show" role="alert">
        <span th:text="${success}"></span>
        <button type="button" class="btn-close" data-bs-dismiss="alert" aria-
label="Close"></button>
    </div>
    <div th:if="${info}" class="alert alert-info alert-dismissible fade show"
role="alert">
        <span th:text="${info}"></span>
        <button type="button" class="btn-close" data-bs-dismiss="alert" aria-
label="Close"></button>
    </div>
    <div th:if="${warning}" class="alert alert-warning alert-dismissible fade
show" role="alert">
        <span th:text="${warning}"></span>
        <button type="button" class="btn-close" data-bs-dismiss="alert" aria-
label="Close"></button>
    </div>
```

```
    <h1 class="text-center mb-3">List of Students</h1>

    <form class="row g-2 mb-3" th:action="@{/students}" method="get">
        <div class="col-auto">
            <input class="form-control" type="text" name="keyword"
placeholder="Search by ID or name" th:value="${keyword}">
        </div>
        <div class="col-auto">
            <button class="btn btn-primary" type="submit">Search</button>
            <a class="btn btn-outline-secondary" th:href="@{/students}">See
All</a>
            <a class="btn btn-success" th:href="@{/students/new}">+ Add</a>
        </div>
    </form>

    <div class="table-responsive">
        <table class="table table-striped align-middle">
            <thead>
            <tr>
                <th>ID</th><th>Name</th><th>Date of
Birth</th><th>Passed</th><th>GPA</th><th class="text-end">Actions</th>
            </tr>
            </thead>
            <tbody>
            <tr th:each="s : ${students}">
                <td th:text="${s.id}"></td>
                <td th:text="${s.name}"></td>
                <td th:text="${#dates.format(s.dob, 'yyyy-MM-dd')}"></td>
                <td>
          <span th:text="${s.passed ? 'Yes' : 'No'}" class="badge"
                th:classappend="${s.passed} ? ' text-bg-success' : ' text-bg-
secondary'"></span>
                </td>
                <td th:text="${s.gpa}"></td>
                <td class="text-end">
                    <a class="btn btn-sm btn-outline-primary"
th:href="@{/students/{id}/edit(id=${s.id})}">Edit</a>
                    <form th:action="@{/students/{id}/delete(id=${s.id})}"
method="post" class="d-inline"
                          onsubmit="return confirm('Delete this record?')">
                        <button class="btn btn-sm btn-outline-danger"
type="submit">Delete</button>
                    </form>
                </td>
            </tr>
            </tbody>
        </table>
    </div>

    <!-- Simple navigation -->
    <div class="d-flex justify-content-between mt-3">

        <!-- Previous button -->
        <a class="btn btn-outline-primary"
           th:if="${start > 0}"
           th:href="@{/students(start=${start-limit}, keyword=${keyword})}">
            Prev
        </a>

        <!-- Next button -->
        <a class="btn btn-outline-primary"
```

```
            th:if="${start + limit < total}"
            th:href="@{/students(start=${start+limit}, keyword=${keyword})}">
             Next
        </a>

    </div>

</section>
</body>
</html>
```

14. Now create the student-form html for update and add.

```
<!doctype html>
<html lang="en" xmlns:th="http://www.thymeleaf.org"
      xmlns:layout="http://www.ultraq.net.nz/thymeleaf/layout"
      layout:decorate="layout">
<head><meta charset="UTF-8"><title>Student Form</title></head>
<body>
<section layout:fragment="content" th:object="${student}">
  <h2 th:text="*{id} == null ? 'Add Student' : 'Edit Student'"></h2>
  <form th:action="${student.id} == null ? @{/students} :
@{/students/{id}(id=*{id})}"
        method="post" >

    <div class="mb-3">
      <label class="form-label">Name</label>
      <input class="form-control" type="text" th:field="*{name}">
      <div class="text-danger small"
th:if="${#fields.hasErrors('name')}" th:errors="*{name}"></div>
    </div>

    <div class="mb-3">
      <label class="form-label">Date of Birth</label>
      <input class="form-control" type="date" th:field="*{dob}">
      <div class="text-danger small"
th:if="${#fields.hasErrors('dob')}" th:errors="*{dob}"></div>
    </div>

    <div class="mb-3">
      <label class="form-label">Passed</label>
      <select class="form-select" th:field="*{passed}">
        <option th:value="true">Yes</option>
        <option th:value="false">No</option>
      </select>
    </div>

    <!-- GPA input -->
    <div class="mb-3">
      <label class="form-label" for="gpa">GPA</label>
      <input id="gpa"
             class="form-control"
             type="number"
             th:field="*{gpa}"
             min="0" max="4.33" step="0.01"
             required
             oninput="validateGpa(this)">
    </div>

    <!-- Script -->
```

```
        <script>
          function validateGpa(el) {
            const min = 0, max = 4.33;
            const val = parseFloat(el.value);

            // If there is a value and it's invalid
            if (el.value && (isNaN(val) || val < min || val > max)) {
              //  Block form submission and show custom error message
              el.setCustomValidity(`Please enter GPA between ${min} and
    ${max}`);
            } else {
              // Clear the error and allow submission
              el.setCustomValidity('');
            }
          }
        </script>


        <div class="d-flex gap-2">
          <button class="btn btn-primary" type="submit">Save</button>
          <a class="btn btn-secondary" th:href="@{/students}">Cancel</a>
        </div>
      </form>
    </section>
    </body>
    </html>
```

15. Run the program. It would look like the following.